

PRD

问题

代码通过 Git 对团队公开，Agent 的工作却困在私人 session 里。它读过什么、知道什么、做过什么判断、下一步是什么，其他人看不到，也无法直接复用。

范式变化

Git 假设作者在系统外，只版本化代码环境。Agent 是系统内会持续积累状态的执行者，因此版本控制对象必须扩展。

代码块

```
1  Git State      = Environment State
2  AgentGit State = Workspace State
3                = Agent State + Environment State + Relations
```

Agent State 解释 Agent 知道什么、为何这样做、做到哪里。Environment State 固定这些判断所基于的 repo、commit 和 stash。Relations 记录 Agent 与 Environment、Agent 与 Agent 之间的工作关系。三者共同组成可版本化的 Workspace State。

核心功能

功能	产品行为
Context Management	从 session 中提取目标、约束、事实、决定、进度和 artifact，形成紧凑、可 diff 的 Agent State
Team Exposure	将 Agent State 发布到 Workspace，团队可以查看、搜索、订阅、比较和追溯
Context Reuse	人或 Agent 可以拉取某个 context revision，在

	Codex、Claude Code 等 runtime 中继续使用
Joint Version Control	Workspace Revision 固定当前 Agent Revision、Environment Revision 和二者关系

团队首先获得一个清楚的 Agent 工作视图。Agent-to-Agent context exchange 是同一套发布和消费接口的进一步使用，不是产品成立的前提。

对象

Agent 和 Environment 是基本对象。Workspace 建立在二者之上，负责组织团队内的 Agent instance、Environment instance、关系和权限。

对象	状态
Agent	身份、规则、skill、tool contract
Agent State	目标、约束、事实与证据、决定、进度、artifact 引用
Environment State	repo identity、commit、stash
Workspace	Agent、Environment、关系和权限的不可变 revision

`stash` 必须覆盖 staged、unstaged 和 untracked changes。每条 context update 必须带来源、时间和 Workspace Revision，避免脱离代码基线传播结论。

交付形式

交付	One-liner
<code>agit</code>	A Git-compatible CLI for versioning Agent Context + Environment.
AgentGit Hub	GitHub for Agent Contexts and Workspaces.

`agit` 保留 Git 的使用习惯。scope 只切换对象库，不改变命令语义：

代码块

```
1  agit <git-args>          = agit -e <git-args> = 在当前 Environment 上执行 Git
2  agit -a <git-args>      = 在当前 Agent Store 上执行同构操作
```

因此已有开发习惯可以直接迁移：

代码块

```
1  agit status                # git status
2  agit commit -am "update code" # 提交 Environment
3  agit push                  # 推送 Environment
4
5  agit -a status              # 查看 Agent Context 变化
6  agit -a add -A
7  agit -a commit -m "update context" # 提交 Agent State
8  agit -a push                # 向团队公开 Context
```

团队协作也直接复用 Git 语义：

目的	命令
----	----

查看其他 Agent 的 Context	<code>agit -a fetch</code> 、 <code>agit -a log</code> 、 <code>agit -a show</code>
合入完整更新	<code>agit -a pull</code> 或 <code>agit -a merge <ref></code>
只采用一项 Context 更新	<code>agit -a cherry-pick <revision></code>
从已有 Context 开始	<code>agit -a checkout -b <name> <ref></code>
比较两个 Context	<code>agit -a diff <ref-a> <ref-b></code>

`agit commit` 固定 Environment Revision，`agit -a commit` 固定 Agent Revision。任一 ref 移动后，`agit` 自动生成 Workspace Revision，记录当前 Agent、当前 Environment 和连边，不增加第三套日常命令。Context 提取与 runtime import/export 由 Adapter 完成。

默认 scope 必须是透明 Git wrapper：参数、退出码、stdout、hook、remote 和 credential helper 均保持兼容。`agit commit -a` 中的 `-a` 仍是 Git 参数；只有出现在 subcommand 前的 `agit -a commit` 才表示 Agent scope。

Hub 提供 Agent Context 首页、Workspace、权限、搜索、订阅、revision、diff 和 history。人可以直接阅读，Agent 可以通过同一接口拉取。第一版是 Registry + Sync，不运行 Agent，也不保存 secret。

统一格式

AgentGit 使用 runtime-neutral 的 `agit/v1`：

代码块

```
1  PortableState = AgentSpecRef
2                      + AgentStateRef
3                      + WorkspaceRevisionRef
```

Codex、Claude Code 等 runtime 只需实现 export、import 和 validate Adapter。完整聊天记录保留为引用，runtime 私有 checkpoint 不能成为跨团队复用的依赖。详细 schema 见 [Codex Session State 调研](./codex-session-state-research.md)。

MVP

第一版保留默认 scope 下完整可用的 Git 操作。团队成员通过 `agit -a commit / push` 发布自己的 Agent Context，通过 `agit -a pull / checkout` 查看并复用其他人的 Context。Workspace 自动记录 Agent Revision 与 Environment Revision 的配对。Codex 和 Claude Code 是首批 Adapter；Agent-to-Agent 自动交换放在这条链路验证之后。

验收

任一团队成员无需询问原作者，就能从 Hub 看清某个 Agent 的目标、已知事实、关键决定、当前进度、artifact 和代码基线，并能用一个命令在自己的 runtime 中复用。Context 更新必须可追溯、可比较，未提交修改不得丢失；对象缺失、版本冲突和 Adapter 不兼容必须显式报告，secret 不得进入 Hub。